PRODUCTION MASTERPLAN

# Mastering Docker Compose: YouTube Production Roadmap

A Complete Multi-Container Orchestration Curriculum, Recording Blueprint, and Packaging Strategy

| COURSE FORMAT | TARGET AUDIENCE | TARGET RETENTION | PROJECT SCOPE |
|---|---|---|---|
| **60–80 Mins** | **Full-Stack & DevOps** | **45%+ AVD** | **4-Tier App** |
| Standalone Masterclass or 4-Part Series | Knows single-container basics | Demo-driven, project-based delivery | Frontend, Backend API, DB & Cache |

## 4-Week Video Production & Delivery Timeline

EXECUTION WORKFLOW

| Phase | Focus Area | Key Actions & Deliverables | Milestone |
|---|---|---|---|
| **Week 1** | Architecture & Demo Setup | Build the reference 4-tier application (Next.js + Fastify/Express + PostgreSQL + Redis). Create broken manual CLI script to show contrast. | Demo Repo Verified |
| **Week 2** | Scripting & Motion Assets | Write timestamped video script with callouts. Design clear architectural SVG/Figma slides showing network isolation, volumes, and health check flows. | Script & Slides Ready |
| **Week 3** | Studio Screencast Recording | Record in 1440p/4K 60fps with clear audio. Capture side-by-side terminal logs ( `docker compose logs -f` ) and intentional failure demos. | A-Roll & B-Roll Locked |
| **Week 4** | Editing, Packaging & Launch | Add terminal zoom cuts, code highlights, and audio pop indicators. Design 3 high-CTR thumbnails and publish companion GitHub repository. | Published & Shared |

## Technical Recording Rig & Viewer Retention Hooks

PRODUCTION QUALITY

### IDE & Terminal Screen Standard

- **VS Code Setup:** Docker & YAML extensions enabled for auto-completion and syntax highlighting.
- **Font Hierarchy:** JetBrains Mono at 18–20pt; line height 1.4 for crisp readability on mobile devices.
- **CLI Contrast:** High-contrast terminal (e.g., Catppuccin Mocha or Tokyo Night) with minimal prompt.
- **Multi-Pane Views:** Top pane: Compose file editing; bottom pane: interactive container log streams.

### YouTube Retention Triggers

- **The First 45 Seconds:** Show the pain of starting 4 containers via raw CLI vs running `docker compose up -d` .
- **Continuous Chapter Progress:** Include a discrete progress bar showing current module topics.
- **Real Failure Demos:** Intentionally trigger race conditions (API starts before DB is ready) before solving with health checks.
- **Pinned Code Base:** Link starter and completed GitHub branches in top comments.

**Pedagogical Approach: Show the Chaos Before the Solution**

Demonstrating how tedious it is to manually manage 4 separate `docker run` commands with manual networks and environment variables creates an immediate "aha!" moment when introducing `docker-compose.yml` .

## Curriculum Breakdown: Modules 1 to 4

FUNDAMENTALS TO ORCHESTRATION

### Module 1: Why Docker Compose? The Multi-Container Problem

Timing: 12 Mins

- **Single Container vs Distributed Apps:** The limitations of running standalone Dockerfiles when dealing with microservices, caching, and databases.
- **Compose Specification Evolution:** Transition from legacy `docker-compose` (v1 Python binary) to modern `docker compose` (v2 Go plugin integrated into Docker CLI).
- **Specification Standards:** Why the `version: '3.8'` key is now obsolete in the latest Compose Specification and how to structure clean manifests.
- **File Naming Conventions:** Default resolution order ( `compose.yaml` , `compose.yml` , `docker-compose.yaml` , `docker-compose.yml` ).

### Module 2: Anatomy of compose.yaml & Core Directives

Timing: 18 Mins

- **Services Block:** Defining individual microservices, base image declarations, and context-based builds ( `build: .` vs `image: redis:alpine` ).
- **Port Mapping vs Port Exposure:** `ports` (host-to-container forwarding) vs `expose` (internal network routing visible only to peer services).
- **Environment Variable Strategies:** Direct inline declaration, `env_file: .env` loading, and default shell variable substitution ( `${VARIABLE:-default}` ).
- **Restart Policies:** Comparing `no` , `always` , `on-failure` , and `unless-stopped` for resilient background operations.

### Module 3: Internal Networking, DNS & Inter-Service Communication

Timing: 15 Mins

- **Automatic Default Bridge:** How Compose provisions a dedicated shared bridge network for each project automatically.
- **Embedded DNS Service Discovery:** Connecting to databases using service names (e.g., `postgres://db:5432/app` ) without manual IP lookup.
- **Network Segmentation:** Creating isolated networks (e.g., public `frontend-net` vs private `backend-net` ) so public services cannot directly reach the database.
- **External Network Aliasing:** Connecting a compose stack into existing shared infrastructure using `external: true` networks.

### Module 4: Storage Management: Persistent Volumes & Bind Mounts

Timing: 15 Mins

- **Ephemeral Database Dilemma:** What happens to database records when containers restart or get pruned.
- **Named Volumes for Data Durability:** Configuring top-level `volumes:` declarations to store database data in Docker-managed disk storage.
- **Bind Mounts for Local Hot Reloading:** Mounting source code into development containers for live updates without rebuilding images on every file save.
- **Volume Backup & Restoration:** Inspecting volumes with `docker volume inspect` and transferring data with temporary export containers.

> **Key Teaching Tip: Visualize the Network Bridge**
>
> Show an animated diagram illustrating how `web` talks to `api` over port 3000, and `api` talks to `db` over port 5432, while `db` is completely hidden from the outside host machine. This clarifies the critical difference between `ports` and `expose` .

## Curriculum Breakdown: Modules 5 & 6

PRODUCTION PATTERNS & TOOLING

### Module 5: Startup Ordering, Health Checks & Compose Watch

Timing: 16 Mins

- **The Startup Race Condition:** Why `depends_on` alone only waits for container creation, not database readiness.
- **Custom Health Checks:** Defining `healthcheck` with `pg_isready` or HTTP pings, interval parameters, and retry counters.
- **Conditional Starts:** Implementing `condition: service_healthy` to ensure downstream microservices wait for complete database initialization.
- **Docker Compose Watch:** Using the modern `develop: watch` feature for instant sync and rebuilds without heavy file sync utilities.

### Module 6: Multi-Environment Overrides & CI/CD Pipelines

Timing: 14 Mins

- **Environment Overrides:** Combining base files with overrides via `docker compose -f compose.yaml -f compose.prod.yaml up -d`.
- **Resource Constraints:** Defining memory and CPU limits in Compose files to prevent noisy neighbor containers.
- **CI Testing Pipeline:** Using Compose inside GitHub Actions to spin up ephemeral test dependencies, execute integration suites, and cleanly tear down.
- **CLI Mastery:** Power commands: `docker compose ps`, `exec`, `top`, `config --resolve-image-digests`, and `down -v --rmi all`.

## YouTube Publishing, SEO & Thumbnail Assets

GROWTH & DISTRIBUTION

### Titles & SEO Strategy

• **Title Option 1:** "Docker Compose Full Course 2026: Multi-Container Mastery"

• **Title Option 2:** "Stop Running Single Containers! Docker Compose Masterclass"

• **High-Volume Tags:** docker compose tutorial, multi container docker, compose watch, full stack docker.

### Thumbnail & Community Engagement

• **Thumbnail Visual:** High-contrast split visual: messy terminal commands vs neat `compose.yaml` icon.

• **Pinned Comment:** Downloadable cheat sheet link + query: "What stack are you orchestrating with Compose?"

• **Shorts/Clips Repurposing:** Clip 1: "Why depends_on breaks your app"; Clip 2: "Compose Watch is magic".

**Creator Action Checklist: Complete Student Repository**

Provide a GitHub repository with step-by-step tags or branches ( `01-starter`, `02-networking`, `03-volumes`, `04-final` ). This prevents viewer drop-off when debugging syntax errors in YAML whitespace.